Problem Statement: E-Commerce Brand Intelligence Agent

Background

AS a seller on e-commerce marketplaces. Before onboarding any new brand as a partner or competitor target, our business team needs a **complete intelligence report** covering:

- Where does the brand currently sell online?
- Who are their sellers on each marketplace?
- Are they running sponsored ads?
- How good are their product listings?
- What are their weaknesses?

Right now, this research is done **manually** — opening each portal, searching for the brand, checking product pages one by one, noting down ratings and seller names, and compiling everything into a spreadsheet. A single brand takes **30–45 minutes** of manual work.

We want this fully automated.

Objective

Build an **AI agent system** that, given a brand name, autonomously:

1. **Discovers** whether the brand is present on Amazon.in and identifies its category.
2. **Extracts** product URLs, ratings, seller names, and ad presence from Amazon.
3. **Evaluates** the quality of the brand's top product listing using a structured audit.
4. **Synthesizes** all collected data into a concise weakness report.
5. **Writes** everything into a structured Excel row and a detailed Markdown report.

The end result is **one row per brand** in an Excel file and a `<brand_name>.md` report file.

What the System Should Do (Functional Requirements)

1. Brand Discovery on Amazon

Given a brand name (e.g., "Mamaearth"), the system should:

- Search for the brand on **Amazon.in** and confirm whether products are listed.
- Identify the **category** and **sub-category** the brand primarily operates in (e.g., Beauty → Face Wash, Health & Nutrition → Protein Powder).

If the brand is not found on Amazon, the system should report this and still produce an Excel row with empty/default values.

2. Product URL Extraction

The system should extract **real, verified product page URLs** from Amazon search results — not guessed or constructed URLs. It should find up to 5 product URLs, prioritized by popularity (rating count).

3. Marketplace Data Collection

For each product URL found, the system should extract:

- **Rating count**: The number of customer ratings (not the star score — the count beside the stars, e.g., "12,453 ratings").
- **Seller name**: Who is selling the product (the "Sold by" value).
- **Sponsored ads**: Whether the brand is running "Sponsored" / paid ads on Amazon search/category pages.

Additionally, the system should identify the **top 1–2 highest-rated product URLs** (by rating count, not star score) for deeper listing quality evaluation.

4. Listing Quality Audit

For the top-rated product URL on Amazon, the system should perform a **listing quality evaluation** (text + images) across 5 dimensions:

Dimension	What it assesses
Title Quality	Is the title long enough? Does it include brand name, sub-category? Does it match the product image?
Visual Quality	How many product images are there? Are they clear?
Content Richness	Are there detailed bullet points? A full product description? Specifications? A+ / rich visual content?
Data Accuracy	Is the pack size correctly stated in both text and images? Do ingredients (if applicable) match between listing and image?

Dimension	What it assesses
Social Proof	What is the average star rating? How many total ratings does the product have?

Each dimension is scored against specific parameters, and the final output is a **1–10 quality score** with a brief remark identifying the listing's weakest areas.

5. Weakness Report

Using all the data collected above, the system should produce a **4–6 bullet point weakness report** that identifies:

- Portal gaps (e.g., "Brand not found on Flipkart, Nykaa, or quick-commerce portals").
- Listing deficiencies (e.g., "Product title is only 3 words — far below the recommended 8+").
- Seller risks (e.g., "Third-party seller detected — brand does not control the Buy Box").
- Visibility issues (e.g., "No sponsored ads running on Amazon").
- Any other concrete weaknesses supported by evidence.

Each bullet should cite evidence from the collected data and be actionable.

6. Outputs

Excel File

All collected data should be written as **one row** in an Excel file (`output.xlsx`) with the following columns:

Column	Description
Brand Name	The input brand name
Category	Top-level category (e.g., Beauty)
Sub-Category	Specific sub-category (e.g., Face Wash)
Portals Live	Which portals the brand is on
Check Ratings	Product URLs with more than 100 customer ratings
Sellers Name	Unique sellers found
Running Ads	Yes/No — does the brand run sponsored ads?
Listing Quality	Score from 1 to 10
Weakness Report	The 4–6 bullet point plain-text report

The Excel file should be **append-only** — each brand run adds a new row without overwriting previous data. Headers should be styled.

Markdown Report

In addition to Excel, the system should generate a **detailed markdown report** saved locally as `<brand_name>.md` (e.g., `Mamaearth.md`). The report should be well-structured and human-readable, covering:

- **Brand Overview**: Portals live, category, sub-category.
- **Product Catalog**: List of discovered product URLs.
- **Marketplace Intelligence**: Sellers, rating counts, ad presence.
- **Listing Quality Audit**: The 1–10 score, a summary of findings, and a breakdown of all evaluation parameters with their individual assessments (GOOD/MODERATE/BAD, YES/NO, etc.).
- **Weakness Report**: The 4–6 bullet point analysis.

Non-Functional Requirements

1. **Autonomous**: Once triggered with a brand name, the system should complete all steps end-to-end without human intervention.
2. **Resilient**: If any individual step fails (e.g., a page doesn't load), the system should gracefully degrade — use defaults (empty lists, zero scores) and continue with remaining steps. It must **never crash** the entire pipeline.
3. **No Hallucination**: Product URLs must come from actual scraped pages. They must never be guessed, constructed from patterns, or invented.
4. **Real Data Only**: Rating counts, seller names — all must be extracted from live pages, not assumed.
5. **Structured Output**: Every piece of data should follow a defined schema. No free-form text in structured fields.
6. **Logging**: The system should print clear, timestamped progress messages to the console so an operator can see what step is running and whether it succeeded or failed.
7. **Scalable**: The system should be able to process multiple brands sequentially by re-running with different brand names, appending each result to the same Excel file.

Technical Requirements

- **Language**: Python 3.12+
- **Package Manager**: `uv`
- **Interface**: CLI only — no frontend required
- **LLM Model**: Any model of your choice

- **AI / Agent Framework**: Any framework of your choice
- **Web Scraping / Browser Automation**: Any library or approach of your choice

Candidates are free to choose their own tools, libraries, and frameworks for implementation. The only hard requirements are Python and `uv`.

Submission Requirements

Along with the working code, candidates must submit:

1. `README.md` — Must include:
    - Candidate's full name
    - Setup instructions (how to install dependencies, configure env, run the agent)
    - A brief summary of the tools/frameworks used and why
    - Any known limitations or trade-offs
    - Sample output (console log snippet + a screenshot of the Excel row or markdown report)
2. **Working code** — Runnable via `uv run main.py "<brand name>"`
3. `.env.example` — A template file listing required environment variables (no real secrets)
4. **Sample outputs** — At least one generated Excel row and one `<brand_name>.md` file

---

Usage

```
uv run main.py "Mamaearth"
```

Expected behavior:

- Console shows step-by-step progress with timestamps.
- `output.xlsx` is created (if not exists) or appended with one new row.
- `<brand_name>.md` is generated with the detailed report.
- Process completes in 2–5 minutes per brand (depending on portal responsiveness).

Evaluation Criteria

When reviewing a candidate's solution, we will check:

Area	Questions we'll ask
Completeness	Do all steps execute? Are both Excel and Markdown files generated?
Correctness	Are product URLs real (from scraped pages) and not fabricated? Are rating counts extracted correctly (count, not stars)?
Resilience	What happens if Amazon blocks a request? Does the pipeline continue or crash?
Data Integrity	Does the listing quality score follow the weighted formula? Are sellers deduplicated?
Code Quality	Is the code organized, readable, and well-structured? Are tools properly documented?
Schema Discipline	Are all outputs validated against defined schemas?

What Success Looks Like

Running `uv run main.py "Mamaearth"` produces:

```
======================================================================
  LEAD GEN AGENT  |  Brand: Mamaearth
  Started: 2025-04-09 14:30:00
======================================================================

  [14:30:05] discovery_agent → found on Amazon | category: Beauty, sub_ca
  [14:31:20] url_extractor → 5 amazon product URLs found
  [14:33:45] marketplace_agent → 6 sellers, running_ads: False
  [14:35:10] listing_quality → score: 7/10
  [14:35:45] weakness_agent → 5 weakness bullets generated
  [14:36:00] excel → saved row 2 to output.xlsx
  [14:36:00] markdown → saved Mamaearth.md

  [14:36:00] ✅ DONE: Mamaearth
======================================================================
```

And two files are produced:

- `output.xlsx` — a complete row with all 9 columns populated.
- `Mamaearth.md` — a detailed report covering brand overview, product catalog, marketplace intelligence, listing quality audit, and weakness analysis.